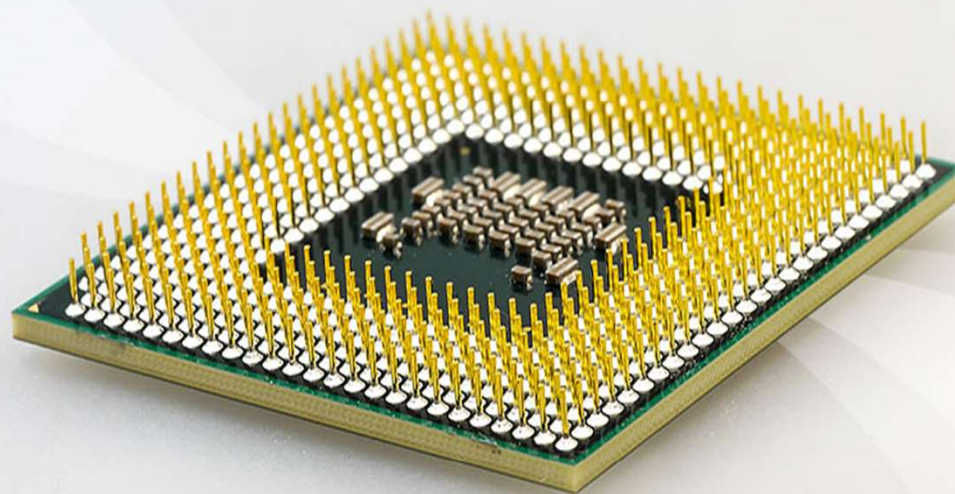


Đại Học Công Nghiệp TP HCM



THIẾT KẾ MẠCH TÍCH HỢP VỚI HDL

GV: Vũ Thị Hồng Nga

Biên soạn năm 2019

Chương 3: Thiết kế mạch tổ hợp với Verilog HDL

Nội dung:

Mô tả các mạch tổ hợp sau dùng Verilog HDL

- Mạch cộng (FA, HA)
- Mạch chọn kênh (MUX)
- Mạch phân kênh (DEMUX)
- Mạch giải mã (DECODER)
- Mạch mã hóa (ENCODER)
- Mạch giải mã led bảy đoạn
- Mạch so sánh
- Mạch nhân
- ALU
- Một số IC thông dụng

Các mô hình Verilog thông dụng:



Mô hình cấu trúc



Mô hình luồng dữ liệu



Mô hình hành vi

I. Mô hình cấu trúc (Structural model)

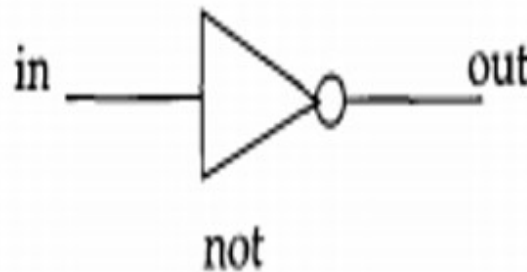
1. Đặc điểm chung:

- Mô hình cấu trúc mô tả các hệ thống dưới dạng các cổng logic hay các khối linh kiện được kết nối lại với nhau để thực hiện được những mạch số mong muốn.
- Mô hình cấu trúc được mô tả một cách trực quan thiết kế số vì mạch logic sau khi tổng hợp sẽ giống với mạch số thông thường
- Mô hình cấu trúc thường áp dụng cho những module nhỏ, không cần tối ưu về timing



2. Các hàm logic được sử dụng trong Verilog

a. Hàm not:



cách gọi hàm: not (out,in)

Ví dụ: Mô tả cổng NOT

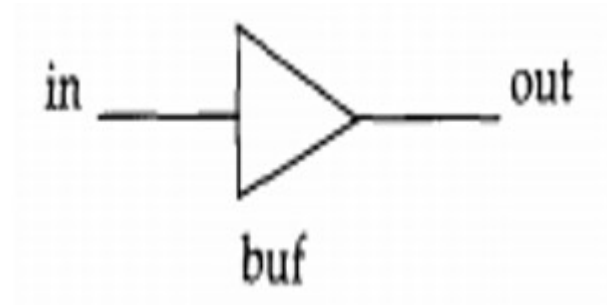
not (out,in) ;

hoặc not U(out,in); //U là tên cổng not



2. Các hàm logic được sử dụng trong Verilog

b. Hàm Buf:



cách gọi hàm: buf (out,in)

Ví dụ: Mô tả cổng buf

buf (out,in)

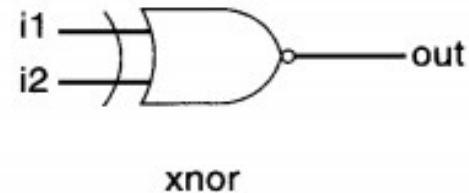
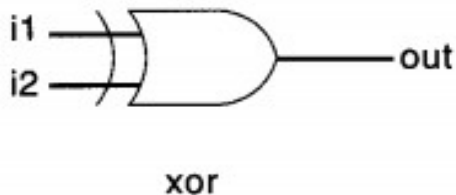
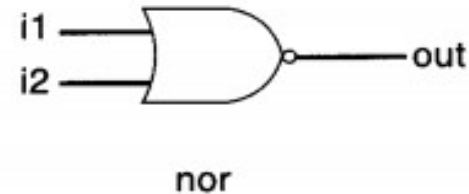
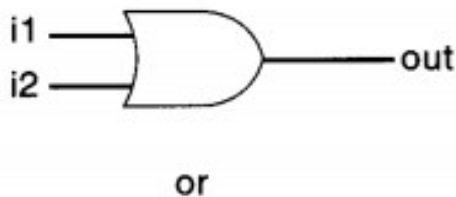
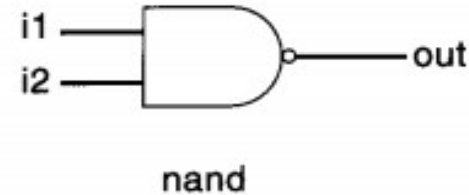
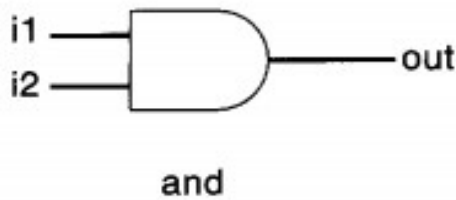
hoặc buf U(out,in) //U là tên cổng buf



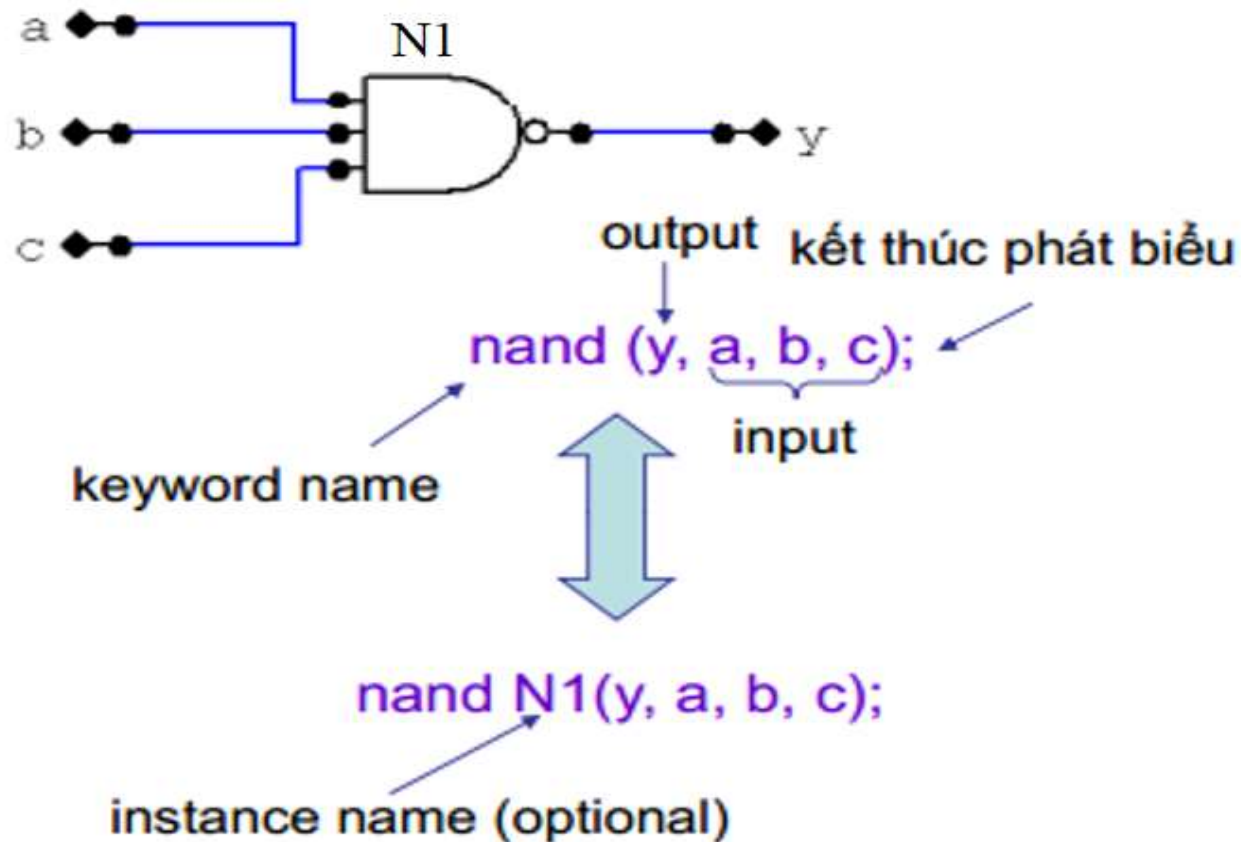
c. Hàm and, nand, or, nor, xor, và xnor

Cách gọi hàm: Mô tả các cổng logic có n input, 1 output

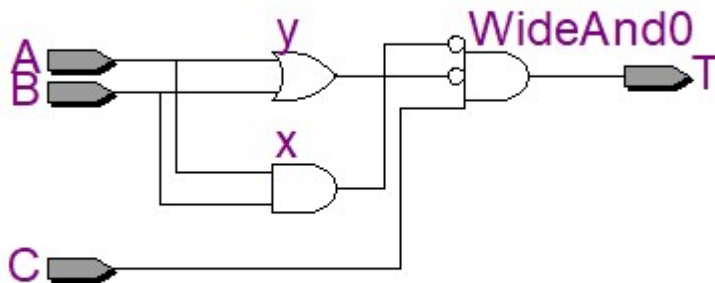
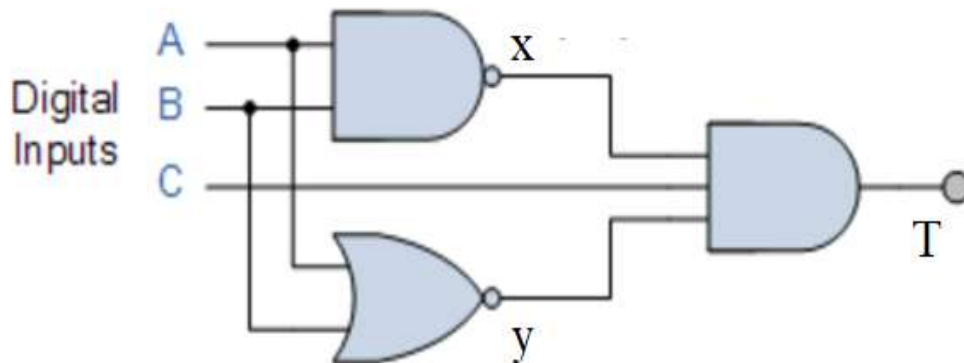
Tên_hàm (output, input1, input2,...inputn)



Ví dụ 1: Mô tả cổng nand ba ngõ vào, một ra



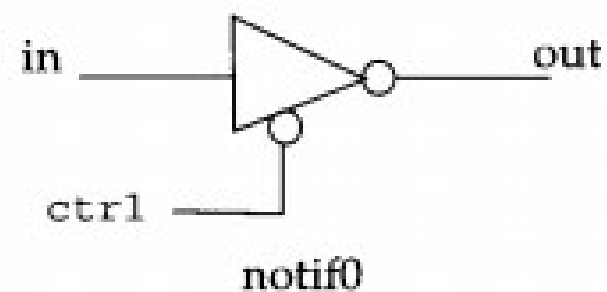
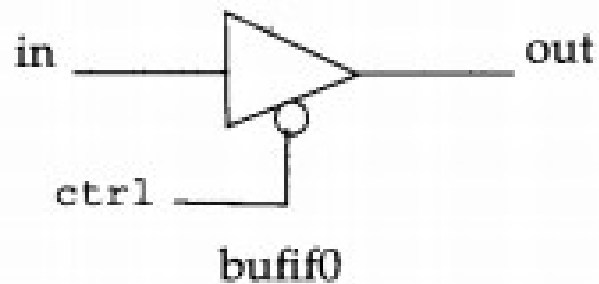
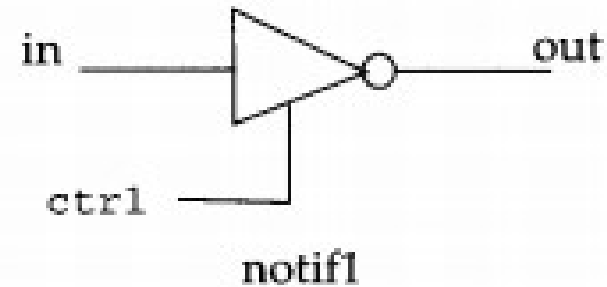
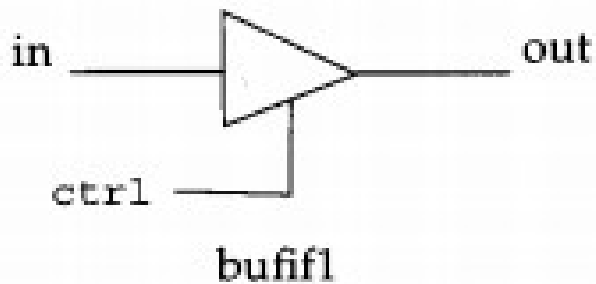
Ví dụ 2: viết chương trình verilog mô tả mạch logic sau dùng mô hình cấu trúc



```
module VD (T, A,B,C);
input A,B,C;
output T ;
wire x,y;
        nand (x,A,B);
        nor (y,A,B);
        and (T,x,y,C);
endmodule
```

c. Bufif / notif

Tên hàm:	bufif1	notif1
	bufif0	notif0



Cách gọi hàm Bufif/notif

bufif1 (out,in,ctrl)

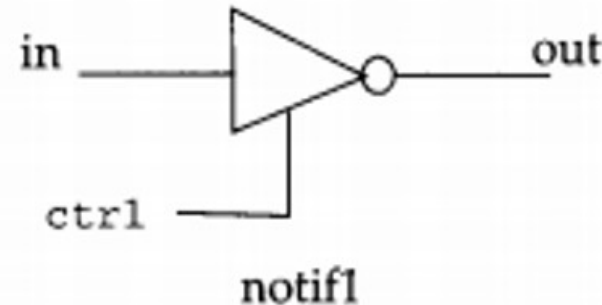
bufif0 (out,in,ctrl)

notif1 (out,in,ctrl)

notif0 (out,in,ctrl)

Ví dụ: Mô tả cổng not có chân cho phép tích cực cao

notif1 (out, in, ctrl)

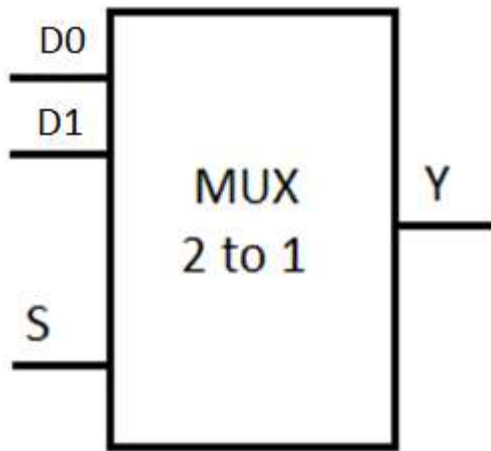


3. Các bước mô tả mạch tổ hợp dùng mô hình cấu trúc

- Xác định số ngõ vào/ngõ ra
- Lập bảng trạng thái
- Viết biểu thức ngõ ra
- Vẽ sơ đồ mạch logic
- Sử dụng các hàm logic có sẵn trong verilog hoặc modul con để mô tả mạch tổ hợp



Ví dụ 3: viết chương trình verilog mô tả MUX2-1 dùng mô hình cấu trúc

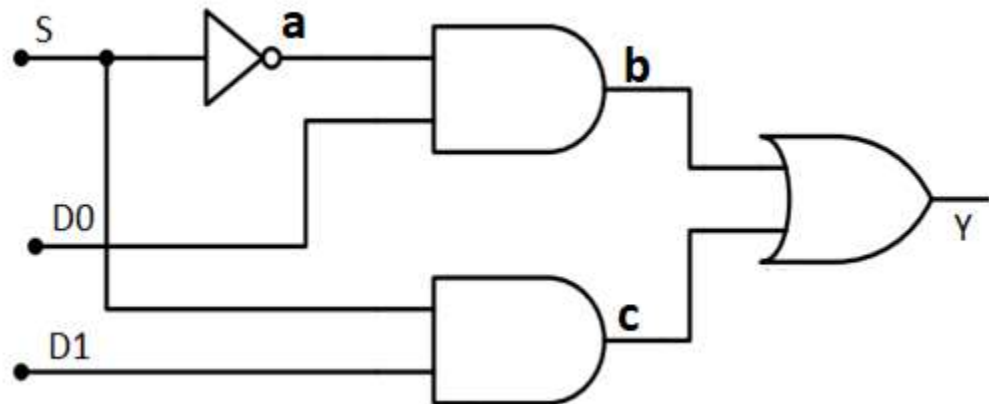


Biểu thức ngõ ra

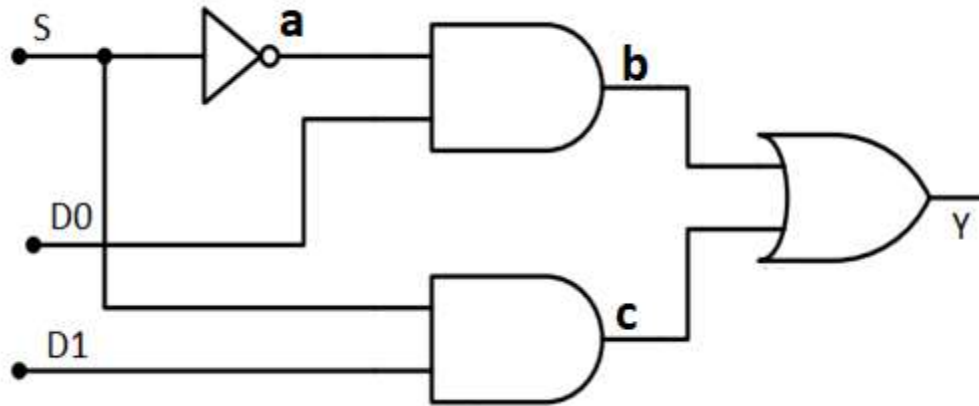
$$Y = \overline{S}D_0 + SD_1$$

Sơ đồ mạch logic

Điều khiển	ngõ ra
S	Y
0	D0
1	D1



THIẾT KẾ MẠCH TÍCH HỢP VỚI HDL

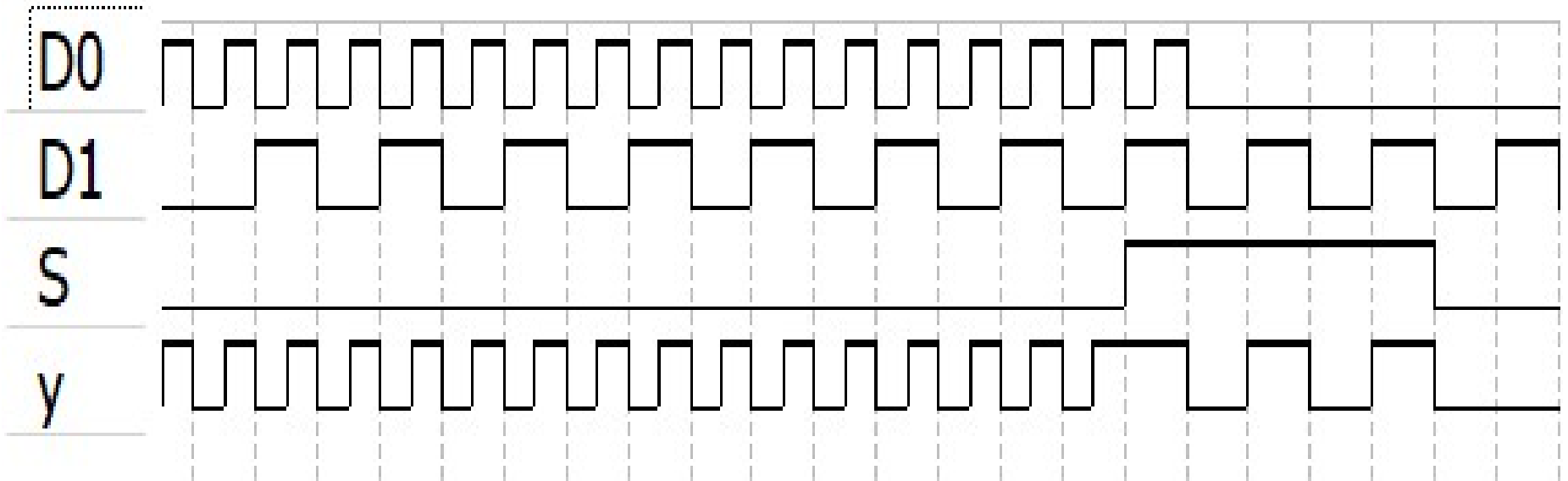


Code Verilog theo mô hình cấu trúc

```
module BT2 (S, D0, D1, y);  
input S, D0, D1;  
output y;  
wire a,b,c;  
    not (a,S);  
    and U1(b,a,D0);  
    and U2(c,S,D1);  
    or (y,c,b);  
endmodule
```



Kết quả mô phỏng



Điều khiển	ngõ ra
S	Y
0	D0
1	D1

Ví dụ 4: viết chương trình verilog mô tả mạch cộng HA dùng mô hình cấu trúc

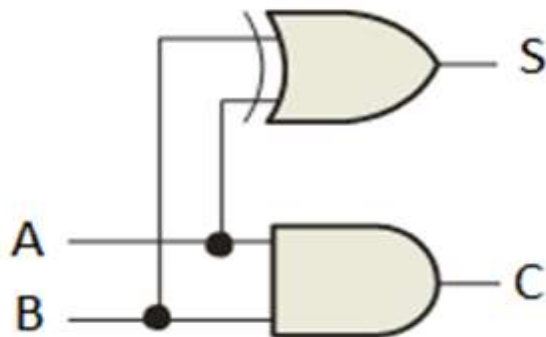
- a. Thiết kế số
 - b. Viết code verilog
- Bài giải:

Biểu thức ngõ ra:

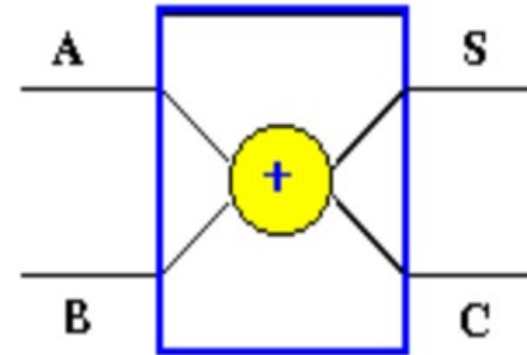
$$S = A \oplus B$$

$$C = A . B$$

Sơ đồ mạch HA:



HA có 2 input, 2 output



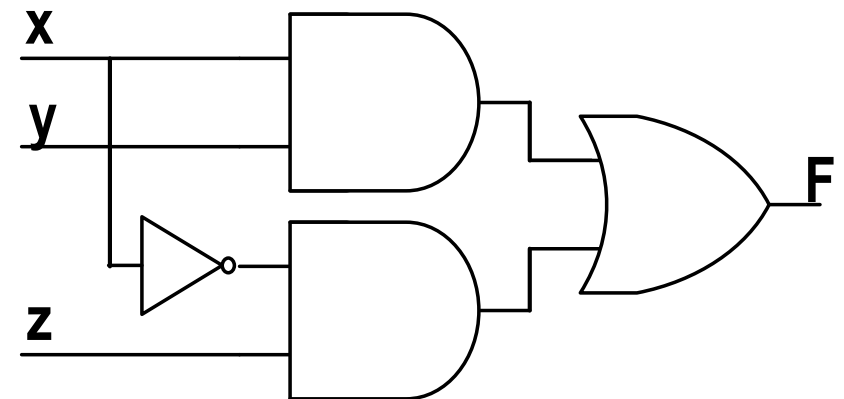
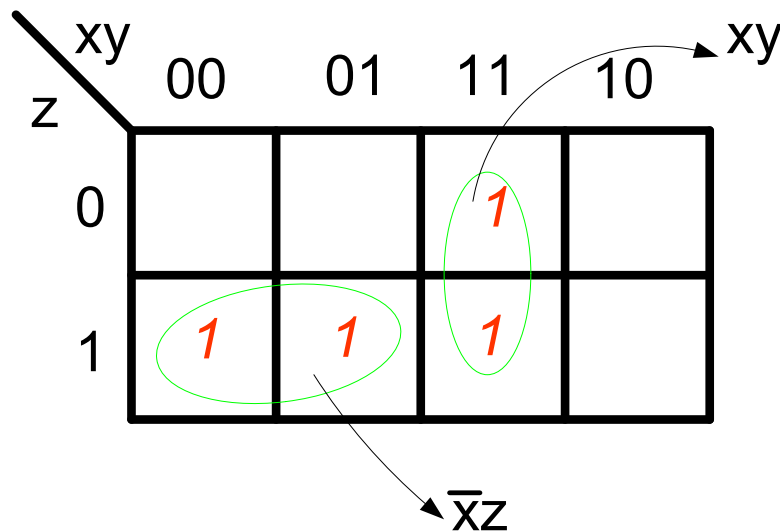
```
module HA (a, b, s,c);  
input a,b;  
output s, c;  
    and (c, a, b);  
    xor (s, a, b);  
endmodule
```



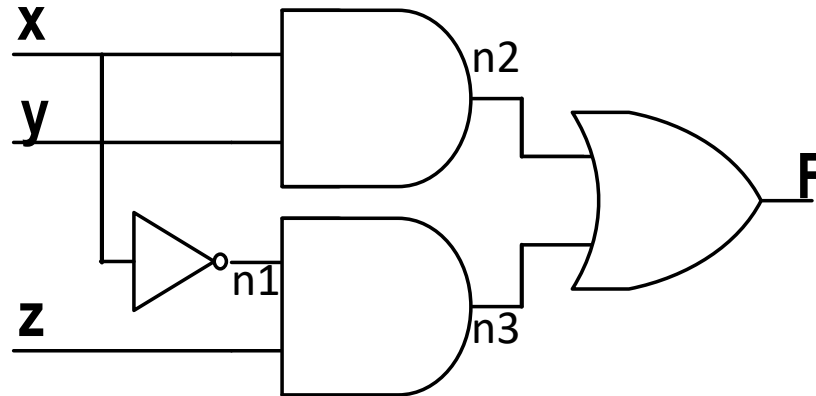
Ví dụ 4: rút gọn hàm $F(x,y,z)$ và viết code mô tả hàm F dùng mô hình cấu trúc

$$F(x, y, z) = \sum m(1,3,6,7) = \bar{x}\bar{y}z + \bar{x}yz + xy\bar{z} + xyz$$

$$F = xy + \bar{x}z$$



THIẾT KẾ MẠCH TÍCH HỢP VỚI HDL



```
module Vidu (F, x,y,z);
input x,y,z;
output F;
wire n1,n2,n3;
    not (n1,x);
    and U1(n2,x,y);
    and U2(n3,n1,z);
    or (F,n3,n2);
endmodule
```

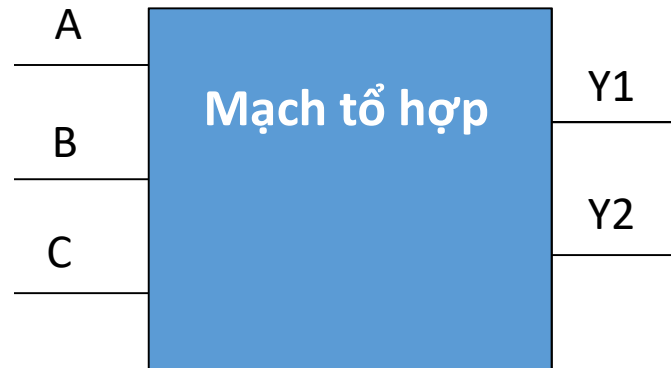


Ví dụ 5: rút gọn hàm $F(x,y,z)$ và viết code mô tả hàm F dùng mô hình cấu trúc

$$f(A,B,C) = \bar{A}.B.\bar{C} + AC + ABC + A$$

- Rút gọn hàm
- Vẽ mạch logic
- Viết code dùng mô hình cấu trúc

Vd 7 : viết code mô tả mạch tổ hợp sau dùng mô hình cấu trúc



A, B, C : ngõ vào

Y1, Y2 : ngõ ra

Y1 = 1 khi số nhị phân ABC khác 0 và chia hết cho 3

Y2 = 1 khi số nhị phân ABC lớn hơn 4

II. Mô hình luồng dữ liệu (DataflowModel)

1. Đặc điểm chung:

- Dataflow Model mô tả luồng dữ liệu bên trong hệ thống số giống như luồng dữ liệu giữa các thanh ghi
- Dataflow Model thường dùng để mô tả mạch tổ hợp thông qua các phép gán liên tục và biểu thức logic
- Dataflow Model thường dùng để mô tả các mạch số vừa và nhỏ



2. Phép gán liên tục

- Phép gán liên tục là phép gán sẽ được thực thi bất cứ khi nào giá trị của biểu thức bên phải thay đổi. Biểu thức bên trái phải có dữ liệu kiểu net.
- Cú pháp:

```
assign biến_wire = biểu_thức;
```

ví dụ 8:

```
module and_gate(a,b,y);  
input a,b;  
output y;  
    assign y = a & b;  
endmodule
```

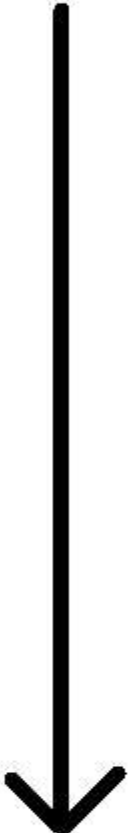


Các toán tử được sử dụng trong dataflow module

Operator Type	Operator Symbol	Operation Performed
Arithmetic	*, /, +, -	As expected
	%	Modulo
Logical	!	Logic NOT
	&&	Logic AND
		Logic OR
Bitwise	~	Bitwise NOT
	&	Bitwise AND
		Bitwise OR
	^	Bitwise XOR
	~^	Bitwise XNOR
Relational	<, >, <=, >=	As expected
Equality	==, !=	As expected
Reduction	&	Red. AND
	~&	Red. NAND
		Red. OR
	~	Red. NOR
	^	Red. XOR
	~^	Red. XNOR
Shift	<<	Left shift
	>>	Right shift
Concat	{ }	Concatenate
Replicate	{ { } }	Replicate
Cond	?:	As expected



Mức độ ưu tiên của các toán tử

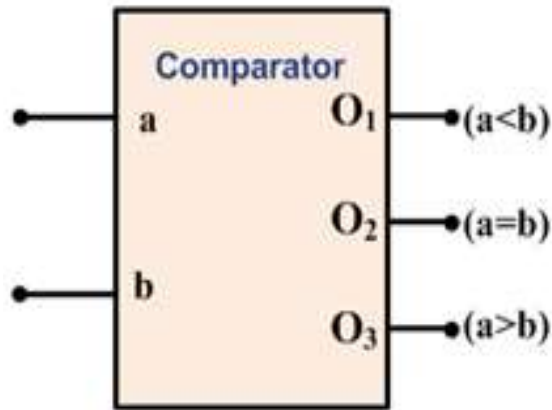
Xử lý logic	Toán tử	Mức ưu tiên
Logic negation and Bit-wise negation (Lấy bù)	! ~	Cao  Thấp
Multiply, divide operations (Nhân, chia)	* / %	
Plus, minus operations (Cộng, trừ)	+ -	
Shift operation (Dịch)	<< >>	
Comparison operation (So sánh hơn)	< <= > >=	
Logical Equality, case equality (sà sánh bằng)	== != === !==	
Bit-wise AND operation	&	
Bit-wise exclusive operation	^ ^~	
Bit-wise OR operation		
Logical AND operation	&&	
Logical OR operation		
Conditional expression (Điều kiện)	? :	

3. Các bước mô tả mạch tổ hợp dùng mô hình luồng dữ liệu

- Xác định số ngõ vào/ ngõ ra
- Lập bảng trạng thái
- Viết biểu thức ngõ ra
- Sử dụng phép gán assign hoặc modul con để mô tả mạch tổ hợp



Ví dụ 9: viết chương trình verilog mô tả mạch so sánh 1 bit dùng mô hình luồng dữ liệu

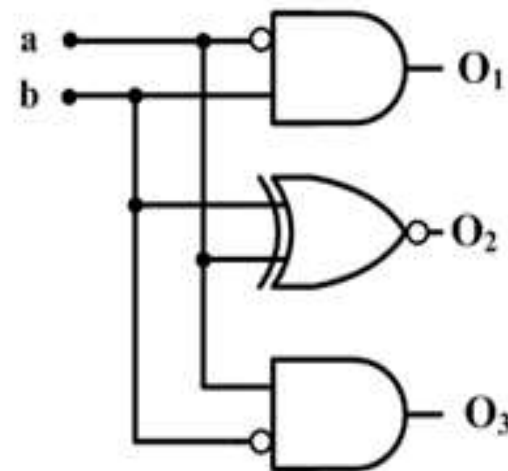


Input		Output		
a	b	O ₁	O ₂	O ₃
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

$$O_1 = \bar{a}b$$

$$O_2 = \overline{a \oplus b}$$

$$O_3 = a\bar{b}$$

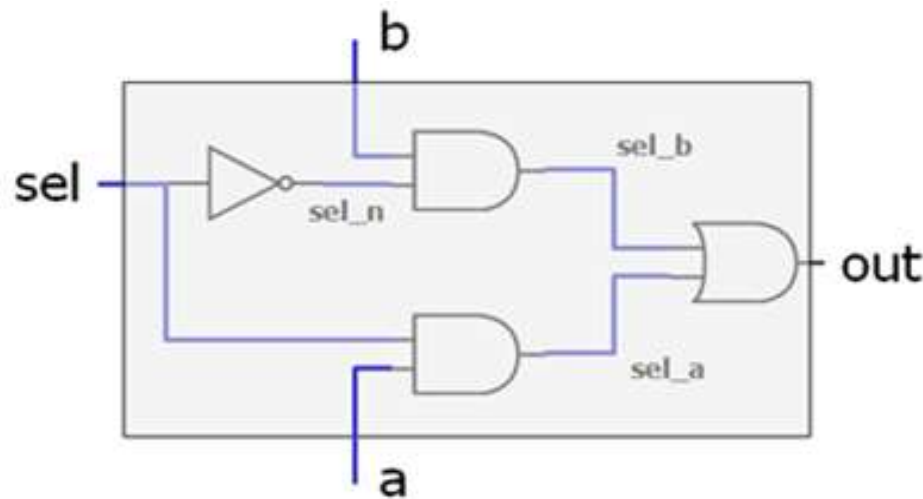


VD10: viết chương trình verilog mô tả mạch so sánh 1 bit dùng mô hình luồng dữ liệu

```
module compare1bit (o1,o2,o3,a,b);  
    input a,b;  
    output o1, o2, o3;  
  
    assign o1 = ~a & b;  
    assign o2 = a ~^ b;  
    assign o3 = a & ~b;  
  
endmodule
```



Ví dụ 11: viết chương trình verilog mô tả mạch tổ hợp sau dùng mô hình luồng dữ liệu

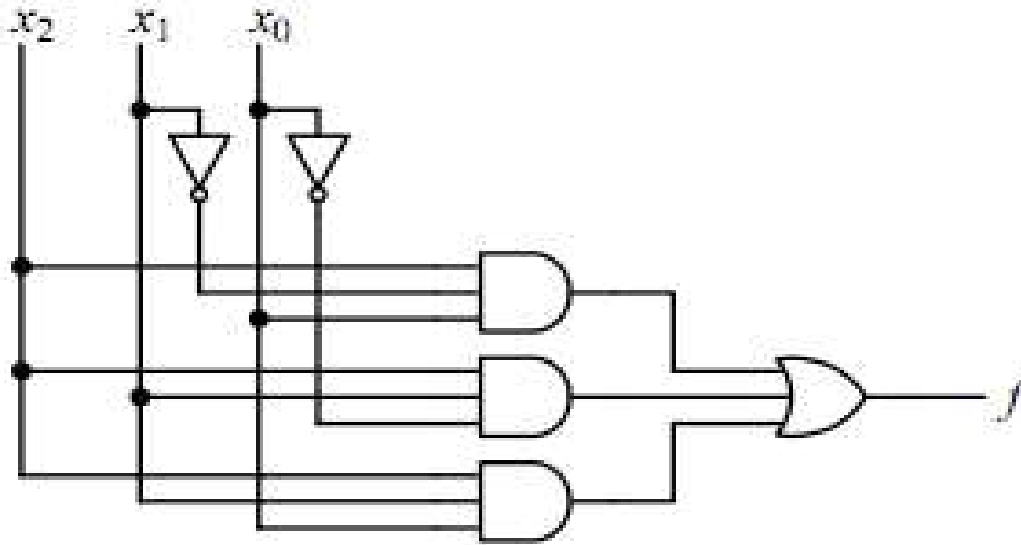


```
module machtohop (out, a, b, sel);  
    input  a, b, sel;  
    output out;  
    assign out = (sel & a) | (~sel & b);  
endmodule
```



Ví dụ 12: viết chương trình verilog mô tả mạch tổ hợp sau dùng mô hình luồng dữ liệu và cấu trúc

Cách 1: mô hình cấu trúc



$$f = x_2x_1'x_0 + x_2x_1x_0' + x_2x_1x_0$$

```
module vd2 (x0,x1,x2,f);
input x1,x2,x0;
output f;
wire n1,n2,n3,n4,n5;
not u1(n1,x1);
not u2(n2,x0);
and u3(n3,x2,n1,x0);
and u4(n4,x2,x1,n2);
and u5(n5,x2,x1,x0);
or u6(f,n3,n4,n5);
endmodule
```

Cách 2: mô hình luồng dữ liệu

```
module vidu_df (x2,x1,x0,f);
input x1,x2,x0;
output f;
assign f = (x2&(~x1)&x0)|(x2&(x1)&(~x0))|(x2&x1&x0);
endmodule
```



Ví dụ 13: viết chương trình verilog mô tả mạch cộng 4 bit dùng mô hình luồng dữ liệu

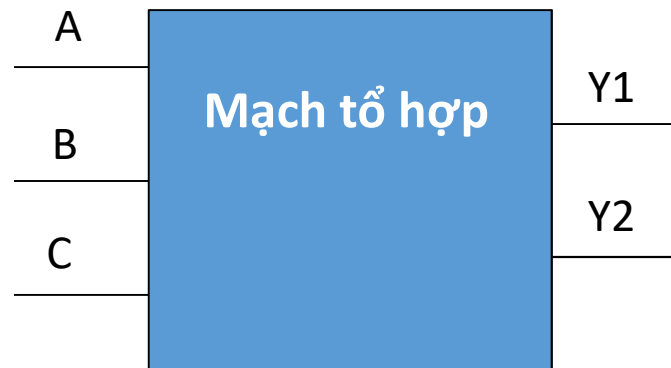
```
module fulladd4(sum, c_out, a, b, c_in);  
    output [3:0] sum;  
    output c_out;  
    input[3:0] a, b;  
    input c_in;  
    assign {c_out, sum} = a + b + c_in;  
endmodule
```



Ví dụ 11: viết chương trình verilog mô tả mạch tổ hợp sau dùng mô hình luồng dữ liệu

ABC	F1	F2
000	1	1
001	0	1
010	0	1
011	1	1
100	0	0
101	0	1
110	0	1
111	0	0

Vd 14 : viết code mô tả mạch tổ hợp sau dùng mô hình luồng dữ liệu



A, B, C : ngõ vào

Y1, Y2 : ngõ ra

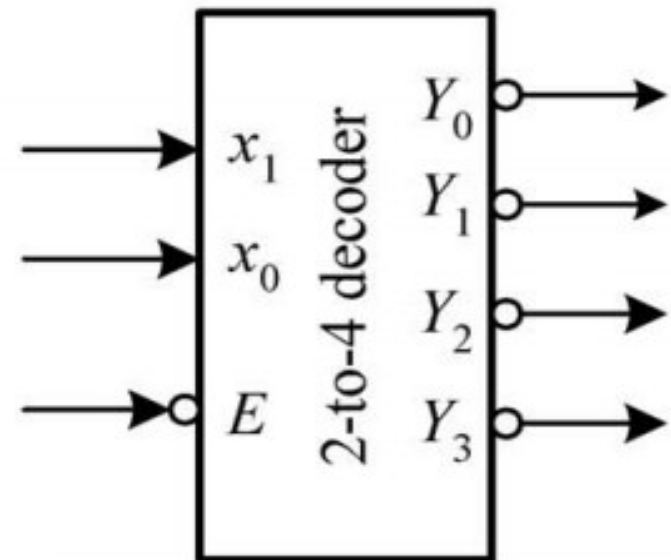
Y1 = 1 khi số nhị phân ABC là số chẵn khác 0

Y2 = 1 khi số nhị phân ABC lớn hơn 5

Ví dụ 15:

Viết chương trình verilog mô tả decoder 2-4 ngõ ra tích cực thấp, chân cho phép tích cực thấp dùng mô hình luồng dữ liệu

- Thiết kế số
- Viết code verilog
- Mô phỏng để kiểm tra kết quả



// ĐÁP ÁN

// decoder 2-4 ngõ ra tích cực thấp, chân cho phép tích cực thấp

```
module decoder_E_df (e, x1, x0, y0, y1, y2, y3);
```

```
input    e, x1, x0;
```

```
output   y0, y1, y2, y3;
```

```
assign   y0 = e | x1 | x0;
```

```
assign   y1 = e | x1 | ~x0;
```

```
assign   y2 = e | ~x1 | x0;
```

```
assign   y3 = e | ~x1 | ~x0;
```

```
endmodule
```



III. Mô hình hành vi (Behavioral model)

1. Đặc điểm chung:

- Mô hình hành vi mô tả hệ thống theo cách mà nó hành xử thay vì kết nối các linh kiện ở mức thấp lại với nhau
- Mô hình hành vi mô tả mối quan hệ giữa các tín hiệu ngõ ra với các tín hiệu ngõ vào mà không cần quan tâm đến cấu trúc phần cứng bên trong nó
- Mức trừu tượng cao hơn và thường để mô tả các mạch số lớn



2. Các bước mô tả mạch tổ hợp dùng mô hình hành vi

- Xác định số ngõ vào/ ngõ ra
- Nắm rõ nguyên lý hoạt động hoặc bảng trạng thái của mạch tổ hợp
- Sử dụng cấu trúc always, if, case, ... để mô tả hoạt động của mạch tổ hợp
- Trong một modul có thể dùng một hoặc nhiều khối always

Lưu ý khi dùng always:

- Các biến nhảy thường là ngõ vào của mạch tổ hợp
- Các biến phải khai báo dạng reg



Ví dụ:

Viết chương trình verilog mô tả decoder 3 to 8, ngõ ra tích cực thấp, chân cho phép (e) tích cực thấp dùng mô hình hành vi

- vẽ sơ đồ chân in/out
- Lập trạng thái
- Viết code
- Mô phỏng và ktra kết quả



E	A_2	A_1	A_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
0	0	0	0	0	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1
0	1	0	1	1	1	1	1	1	0	1	1
0	1	1	0	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	0
1	x	x	x	1	1	1	1	1	1	1	1

```
module decoder (A,E,Y);
```

```
input [2:0]A;
```

```
input E;
```

```
output [7:0]Y;
```

```
reg [7:0]Y;
```

```
always @ (A,E) begin
```

```
    if (E == 1'b0)
```

```
        begin
```

```
            case (A)
```

```
                3'b000: Y= 8'b11111110;
```

```
                3'b001: Y = 8'b11111101;
```

```
                3'b010: Y = 8'b11111011;
```

```
                3'b011: Y = 8'b11110111;
```

```
                3'b100: Y = 8'b11101111;
```

```
                3'b101: Y = 8'b11011111;
```

```
                3'b110: Y = 8'b10111111;
```

```
                3'b111: Y = 8'b01111111;
```

```
            endcase
```

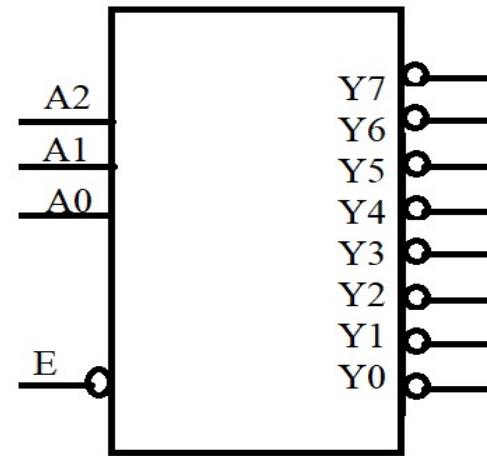
```
        end
```

```
    else
```

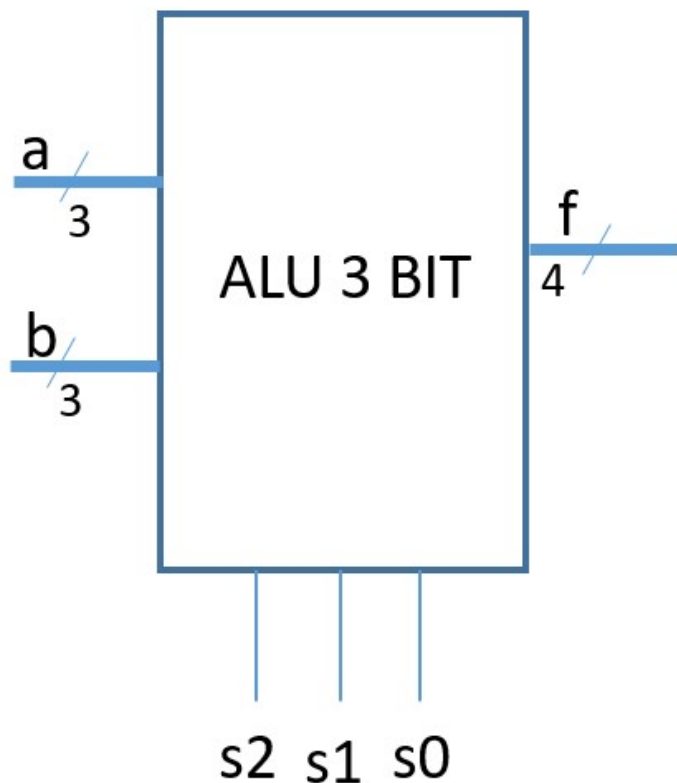
```
        Y = 8'b11111111;
```

```
end
```

```
endmodule
```



Ví dụ: viết chương trình verilog mô tả ALU 3 bit với 8 chức năng dùng mô hình hành vi



s2	s1	s0	f
0	0	0	a
0	0	1	a and b
0	1	0	a or b
0	1	1	not (a)
1	0	0	a + b
1	0	1	a - b
1	1	0	a + 1
1	1	1	a - 1



```
module alu_bh (a, b,s, f);  
input [2:0]a, b,s;  
output [3:0]f;  
reg [3:0]f;  
always @ (a, b, s) begin  
    case (s)  
        3'b000 : f = a;  
        3'b001 : f = a&b;  
        3'b010 : f = a|b;  
        3'b011 : f = ~a;  
        3'b100 : f = a+b;  
        3'b101 : f = a-b;  
        3'b110 : f = a + 1'b1;  
        3'b111 : f = a - 1'b1;  
    endcase  
end  
endmodule
```



VI. Thiết kế mạch số theo cấu trúc phân cấp

- Verilog hỗ trợ cấu trúc phân cấp bằng cách cho phép modules được nhúng trong modules khác.
- Modules cấp độ cao hơn tạo thể hiện của module ở cấp độ thấp và giao tiếp với chúng thông qua các đầu vào, đầu ra và đầu vào ra hai chiều.
- Cấu trúc phân cấp giúp người thiết kế chia một hệ thống thiết kế ra thành các module nhỏ hơn để dễ thiết kế và kiểm soát luồng dữ liệu trong quá trình thiết kế.

a. Nguyên tắc thiết kế theo cấu trúc phân cấp

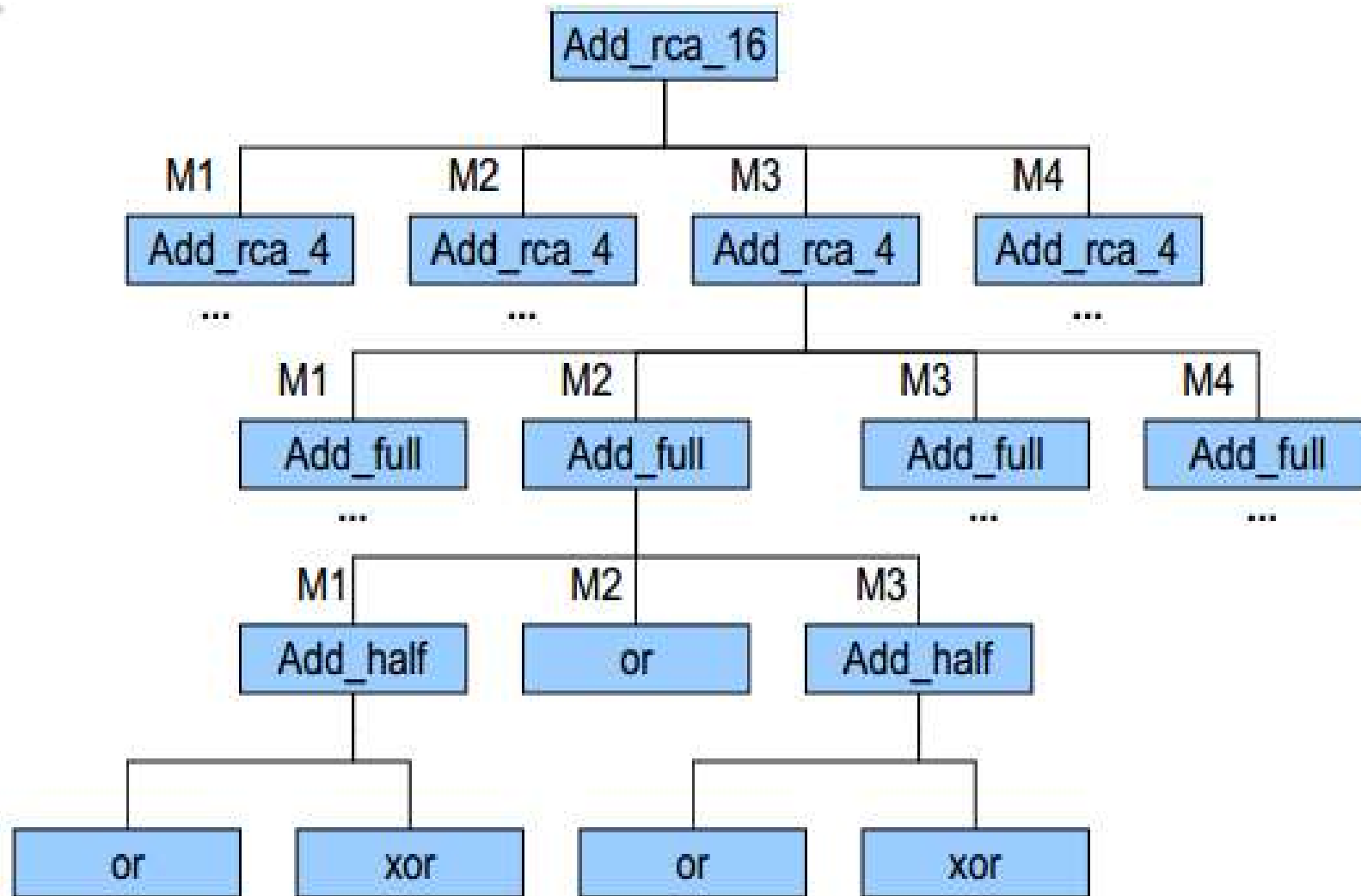
- top-level module là module ở cấp cao nhất
- module thấp nhất chứa các hàm được định nghĩa sẵn trong verilog
- các module được viết trong một hoặc nhiều file khác nhau
- module cấp cao muốn gọi module cấp thấp hơn thì trong chương trình của nó phải có lệnh gọi module thấp đó

b. Module con

- Là module được định nghĩa lồng trong một module khác
- Lệnh gọi và gán đặc tính cho module con sẽ tạo ra một hoặc nhiều bản sao của module được định nghĩa
- Cách gọi module con như sau:

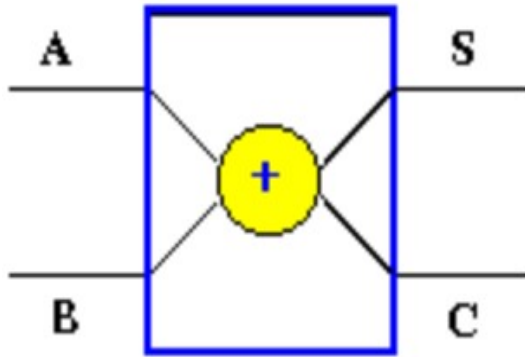
```
module_name_instance    name (port_connection_list);
```

VD: Cấu trúc phân cấp của bộ cộng 16 bit



Ví dụ 1: viết code Verilog mô tả mạch **FA** dùng **HA**

Mạch cộng HA



Chức năng: HA cộng hai số nhị phân A và B một bit không có nhớ

Biểu thức ngõ ra:

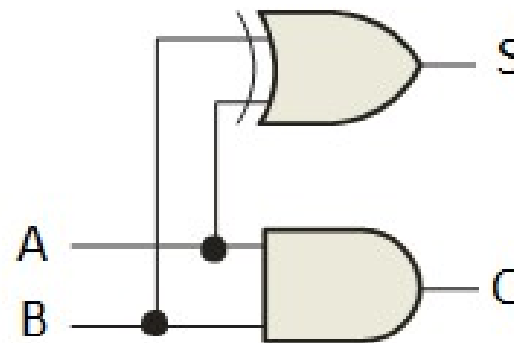
$$S = A \oplus B$$

$$C = A . B$$

Bảng trạng thái:

Vào		Ra	
<i>A</i>	<i>B</i>	<i>S</i>	<i>C_o</i>
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sơ đồ mạch HA:



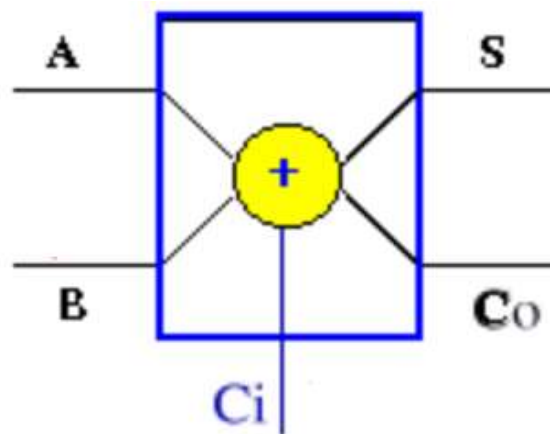
Mạch cộng FA

Chức năng:

FA cộng hai số nhị phân A và B một bit có nhớ.

Bảng trạng thái:

Vào			Ra	
A	B	C_i	S	C_o
0	0	0	0	0
1	0	0	1	0
0	1	0	1	0
1	1	0	0	1
0	0	1	1	0
1	0	1	0	1
0	1	1	0	1
1	1	1	1	1

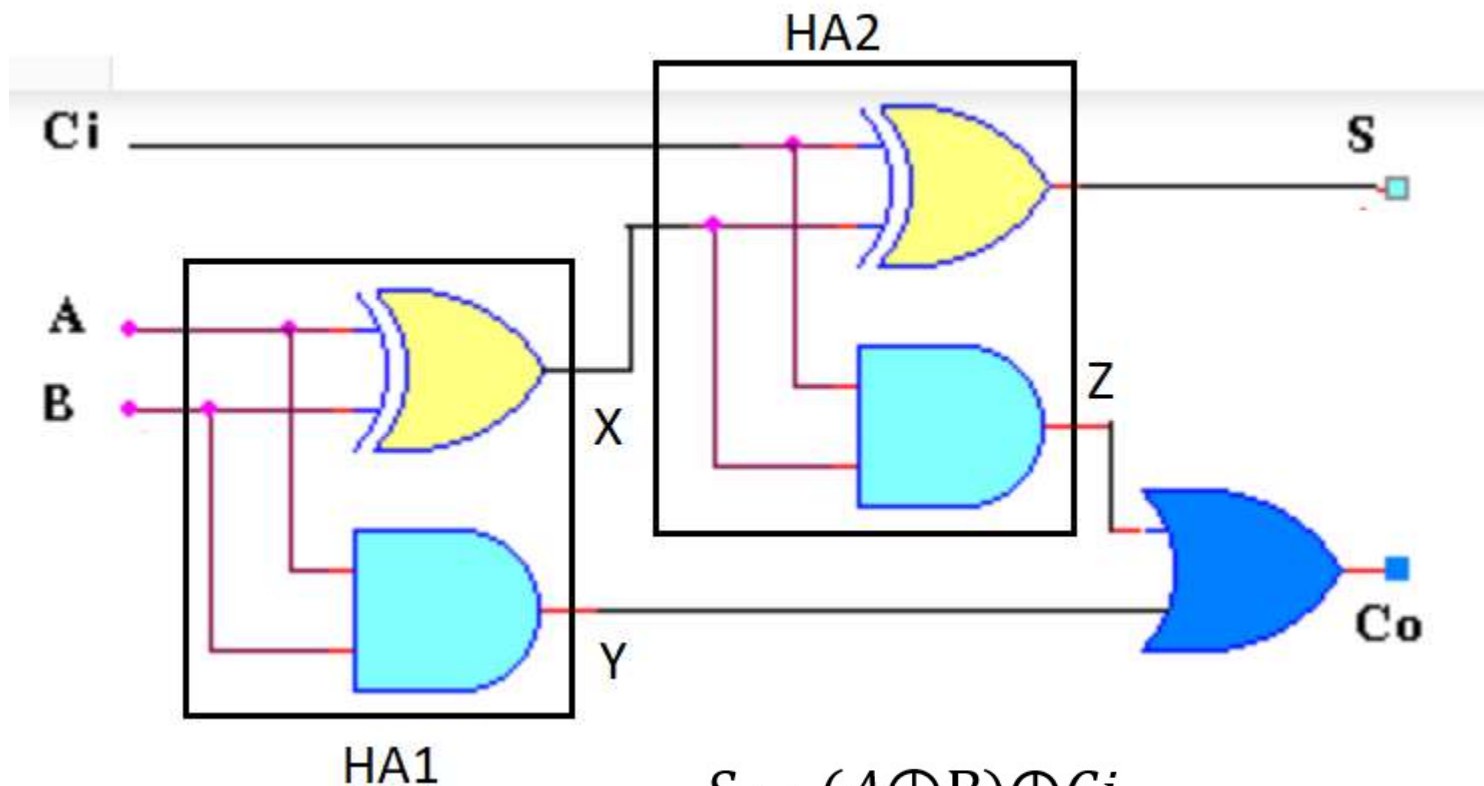


Biểu thức ngõ ra:

$$S = (A \oplus B) \oplus C_i$$

$$C_o = AB + (A \oplus B)C_i$$

Sơ đồ phần cứng mạch cộng FA dùng HA



$$S = (A \oplus B) \oplus C_i$$

$$C_o = AB + (A \oplus B)C_i$$

Code mô tả mạch cộng FA dùng HA

```
module fa_hier (s,co,ci,a,b); //module top
```

```
input ci, a,b;
```

```
output s,co;
```

```
wire x,y,z;
```

```
ha HA1(x,y,a,b);
```

```
ha HA2(s,z,ci,x);
```

```
or (co,z,y);
```

```
endmodule
```

```
//module instance
```

```
module ha (s,c,a,b);
```

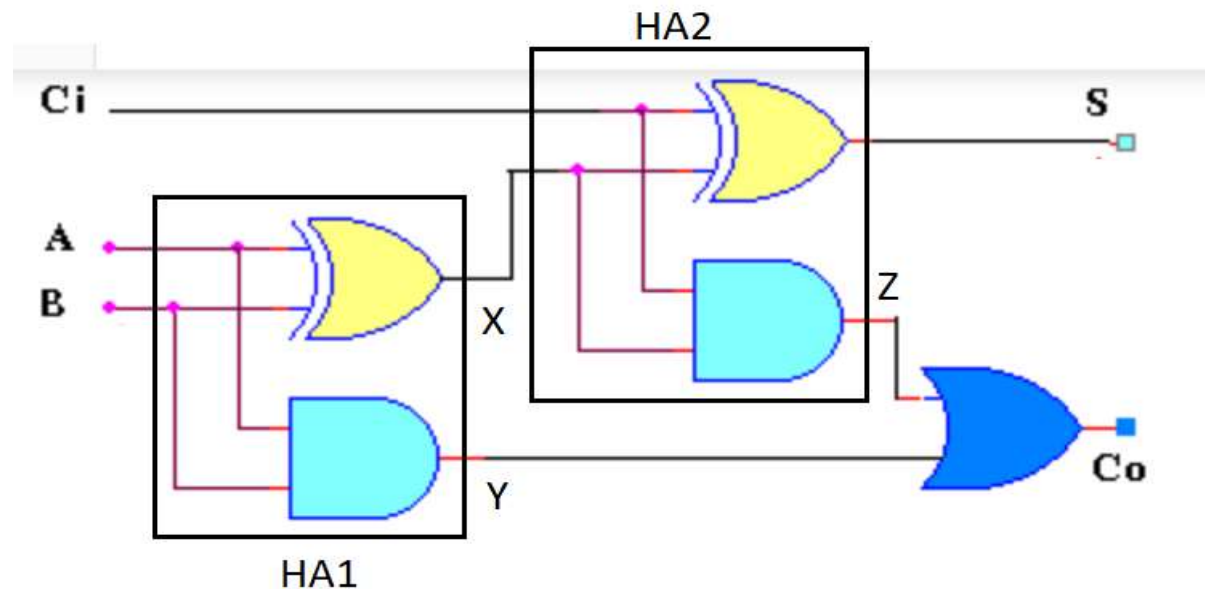
```
input a,b;
```

```
output s,c;
```

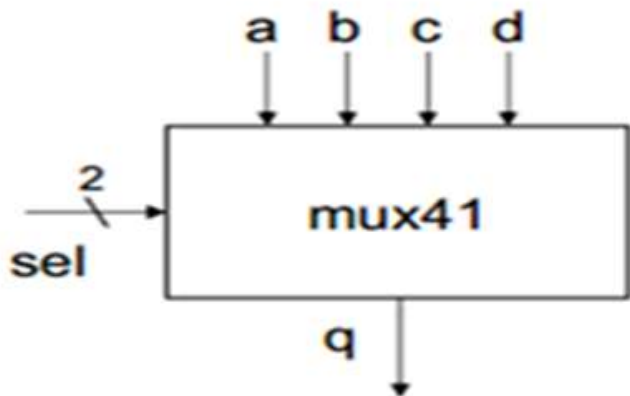
```
assign s = a ^ b;
```

```
assign c = a & b;
```

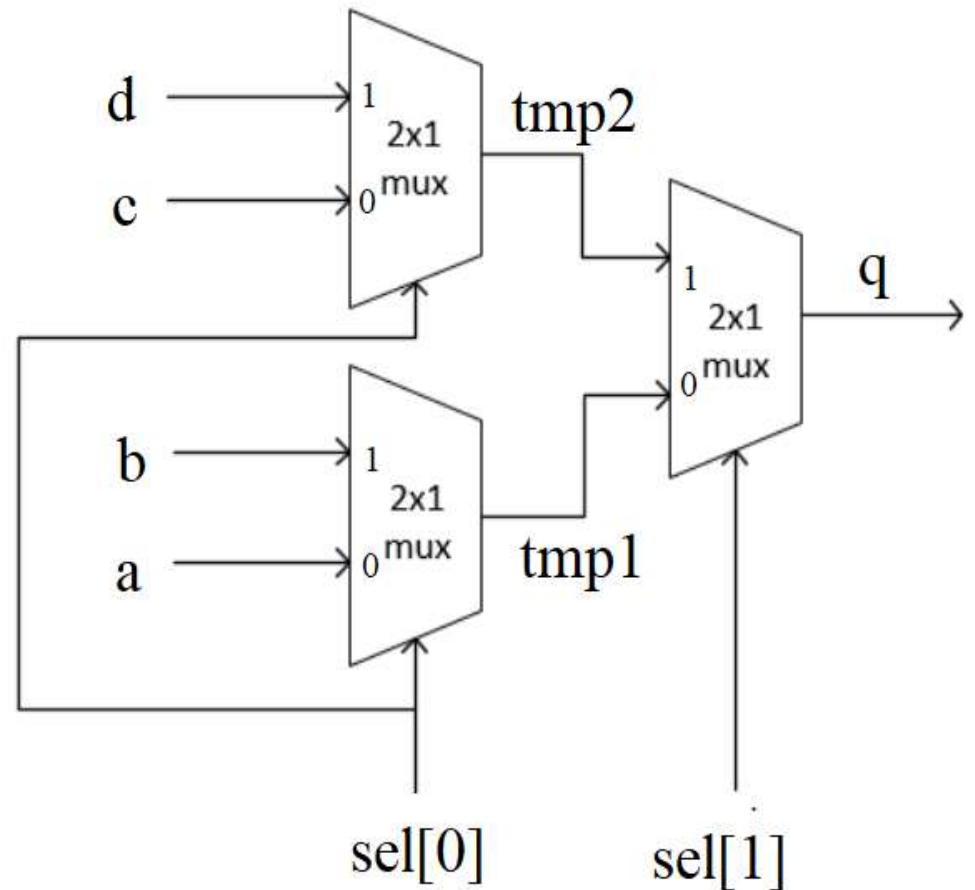
```
endmodule
```



Ví dụ 2: viết chương trình verilog mô tả Mux 4 to 1 sử dụng Mux 2 to 1



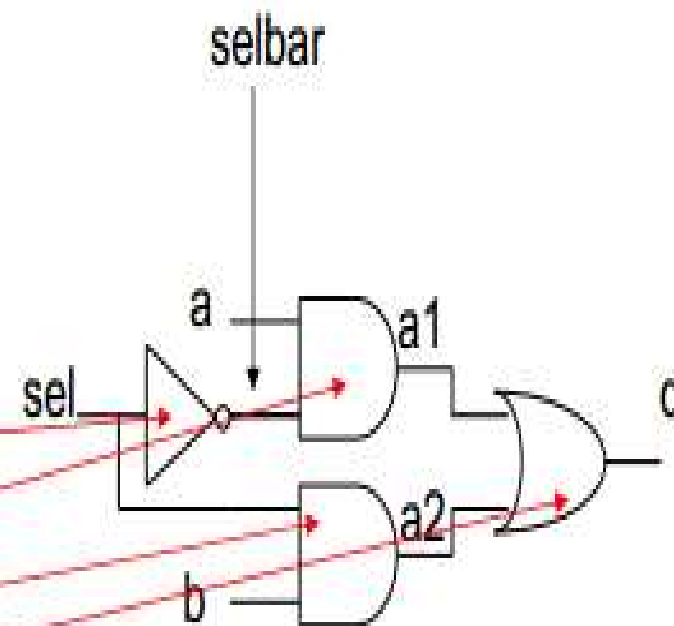
sel	q
00	a
01	b
10	c
11	d



Đáp án:

- Viết module con Mux2to1

```
module mux21(q, sel, a, b);  
    input sel, a, b;  
    output q;  
    wire selbar, a1, a2;  
  
    not(selbar, sel);  
    and(a1, selbar, a);  
    and(a2, sel, b);  
    or(q, a1, a2);  
  
endmodule
```



Viết code MUX4to1 dùng MUX 2to1

```
//module top
```

```
module mux4to1(q,sel,a,b,c,d);
```

```
  input [1:0] sel;
```

```
  input a,b,c,d;
```

```
  output q;
```

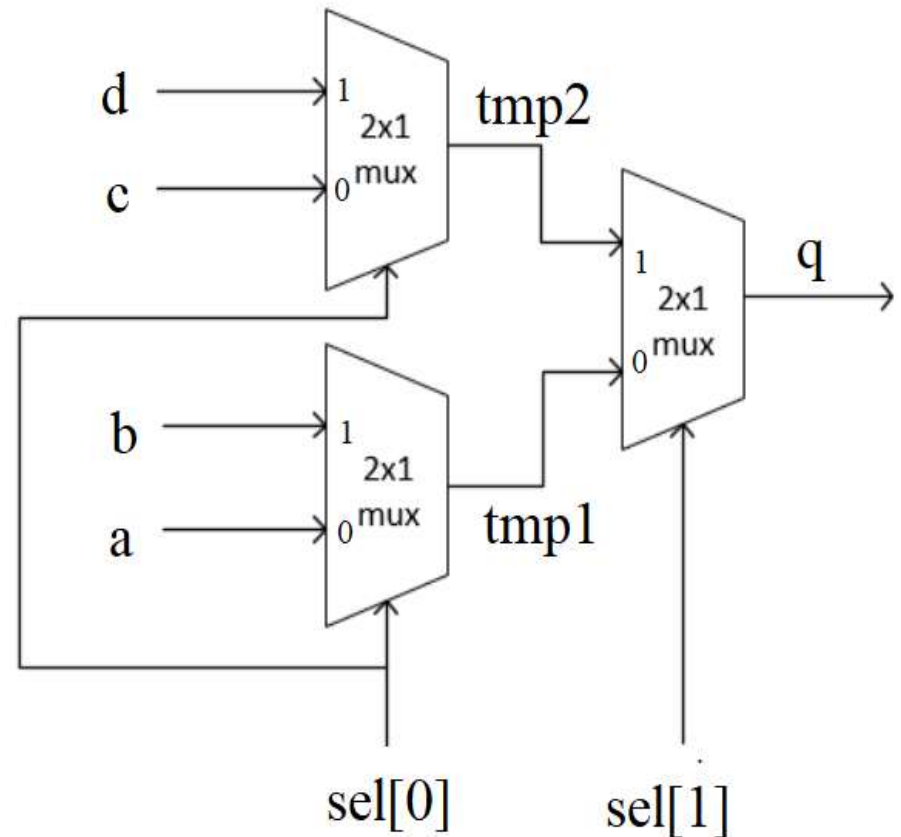
```
  wire tmp1,tmp2;
```

```
  mux2to1 M0 (tmp1,sel[0],a,b);
```

```
  mux2to1 M1 (tmp2,sel[0],c,d);
```

```
  mux2to1 M2 (q,sel[1],tmp1,tmp2);
```

```
endmodule
```



code mô tả MUX 4 to 1 dùng MUX 2to1

//module top

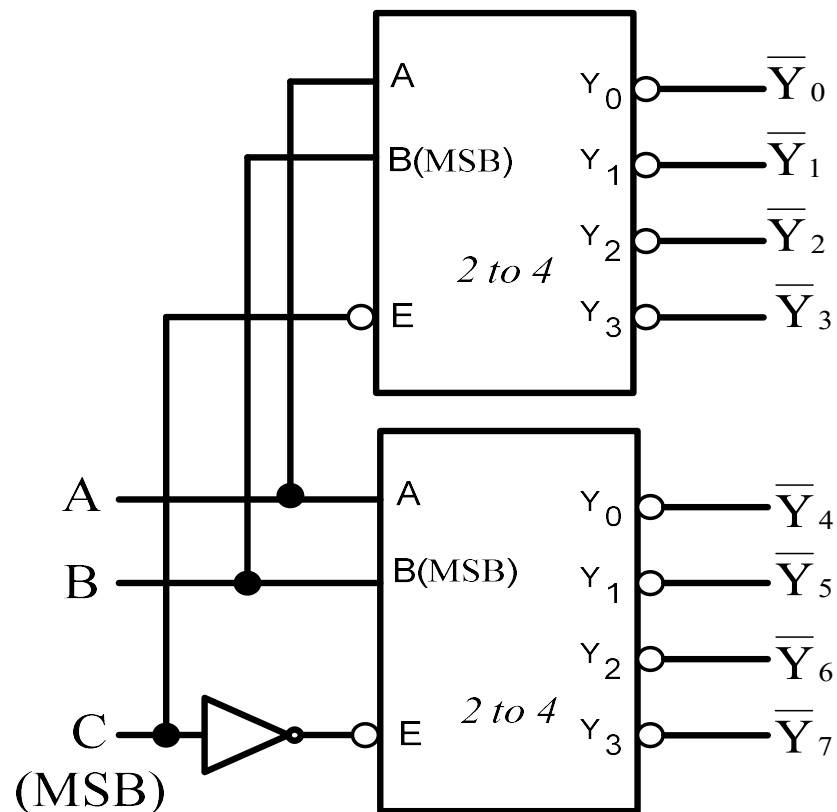
```
module mux4to1(q,sel,a,b,c,d);  
  input [1:0] sel;  
  input a,b,c,d;  
  output q;  
  wire tmp1,tmp2;  
  
  mux2to1 M0 (tmp1,sel[0],a,b);  
  mux2to1 M1 (tmp2,sel[0],c,d);  
  mux2to1 M2 (q,sel[1],tmp1,tmp2);  
endmodule
```

//module instantiation

```
module mux2to1(q,s,a,b);  
  input s,a,b;  
  output q;  
  wire selbar,a1,a2;  
  
  not(selbar,s);  
  and (a1,selbar,a);  
  and (a2,s,b);  
  or (q,a1,a2);  
endmodule
```

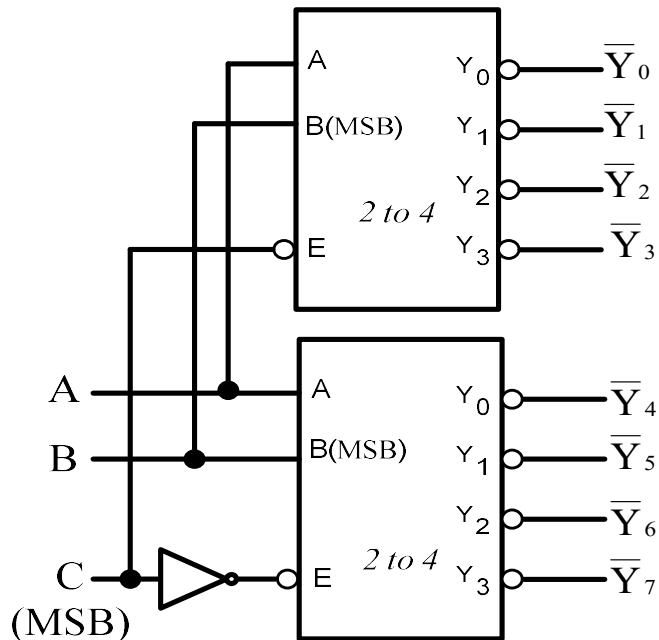


Ví dụ 3: viết chương trình verilog mô tả decoder 3→8
ngõ ra tích cực thấp dùng decoder 2→4



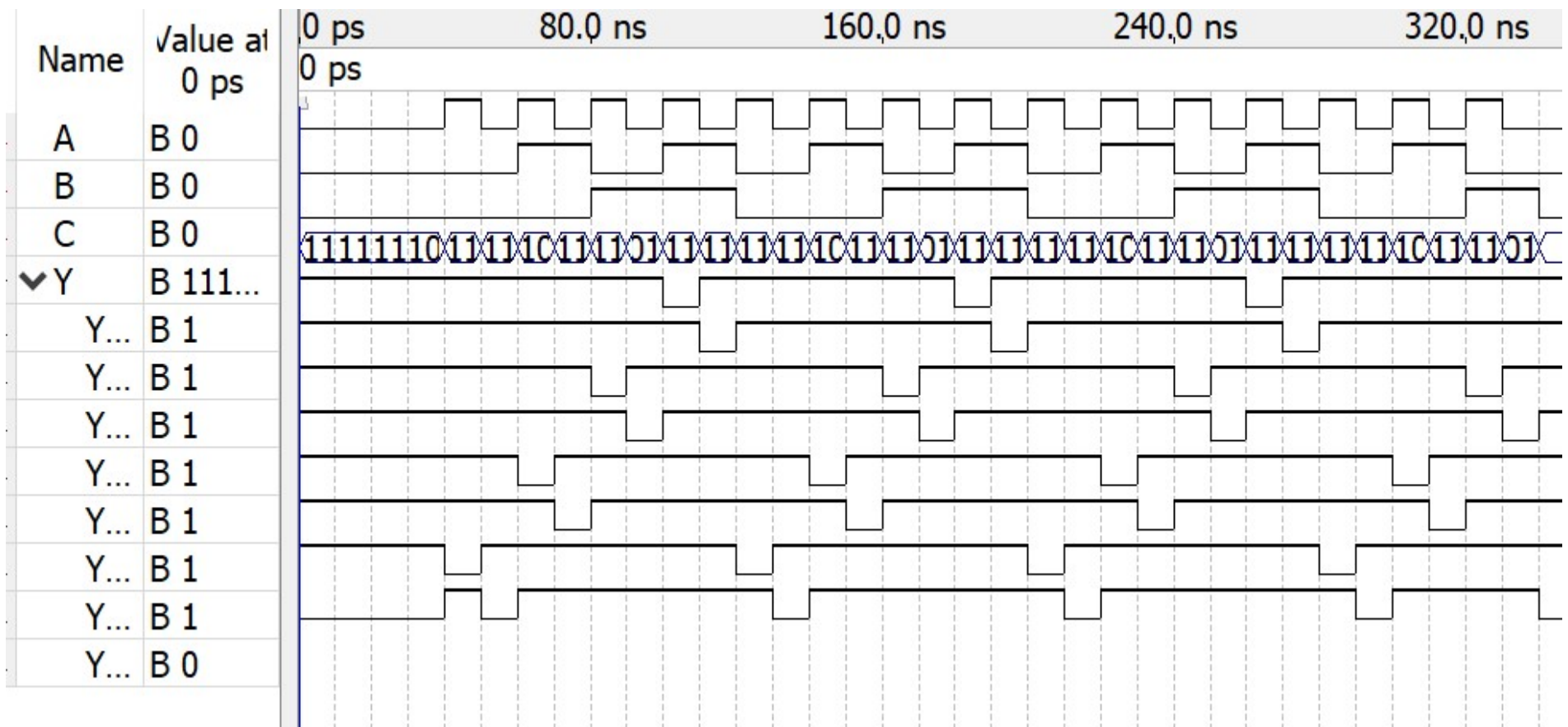
Đáp án

```
module decoder3_8 (A,B,C,Y);
input A,B,C;
output [7:0]Y;
wire notc;
not (notc,C);
decoder2_4 u1(B,A,notc,Y[7:4]);
decoder2_4 u2(B,A,C,Y[3:0]);
Endmodule
```

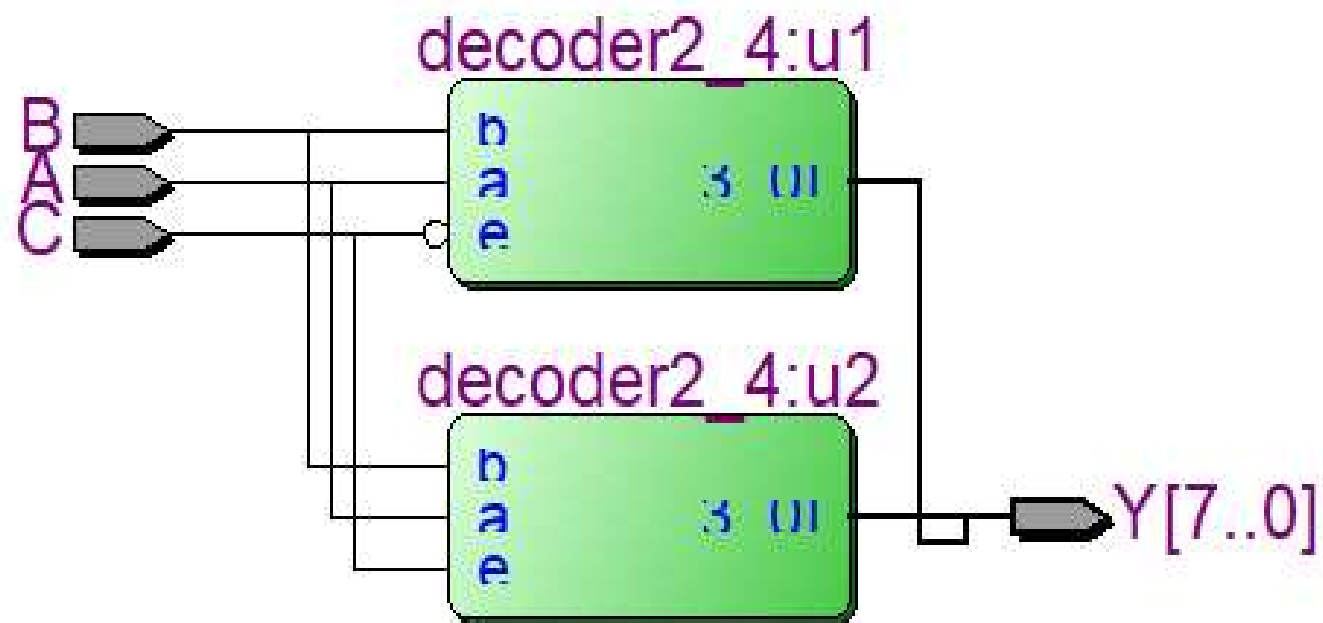


```
module decoder2_4(b,a,e,y);
input a,b,e;
output reg [3:0]y;
always @ (a,b,e)
begin
    if (e == 1'b1)
        y = 4'b1111;
    else
        begin
            case ({b,a})
                2'b00: y = 4'b1110;
                2'b01: y = 4'b1101;
                2'b10: y = 4'b1011;
                2'b11: y = 4'b0111;
            endcase
        end
end
endmodule
```

Kết quả mô phỏng
C là MSB



Kết quả chạy nestlist





Thank you for
listening

